

PE-EC603D

Information Theory and Coding

Combined Module Notes

Modules 1 – 3

Semester VI • 3L:0T:0P • 3 Credits

CGEC | B.Tech ECE (2023–27)

Biraj Sarkar

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Contents

Module 1: Basics of Information Theory	4
1 Basics of Information Theory	4
1.1 Model of a Digital Communication System	4
1.2 Measure of Information Content	4
1.3 Entropy for Discrete Ensembles	5
1.4 Properties of Entropy	5
1.4.1 Derivation of Maximum Entropy Condition (Proof of Maximality)	5
1.5 Binary Entropy Function	6
2 Joint, Conditional, and Mutual Information	7
2.1 Definitions of Entropies	7
2.1.1 Derivation of Joint Entropy Relations	8
2.2 Mutual Information	8
2.3 Properties of Mutual Information	8
2.4 Entropy Relationship Diagram	9
3 Shannon's First Theorem & Source Coding	9
3.1 Shannon's Noiseless Coding Theorem	9
3.2 Performance Metrics of Source Codes	9
3.3 Prefix Codes and Kraft-McMillan Inequality	10
3.4 Shannon-Fano Coding Algorithm	10
Module 2: Markov Sources	16
1 Markov Sources	16
1.1 Definition of Markov Sources	16
1.2 Transition Probability Matrix (TPM)	16
1.3 Stationary State Probabilities	17
1.4 Entropy of Markov Sources (Entropy Rate)	17
2 Shannon's Second Theorem (Noisy Channel Coding Theorem)	18
2.1 Channel Capacity Concept	19
2.2 Noisy Channel Coding Theorem	19
3 Calculation of Channel Capacity for Discrete Channels	19
3.1 Binary Symmetric Channel (BSC)	19
3.1.1 Derivation of BSC Channel Capacity	20
3.2 Binary Erasure Channel (BEC)	20
3.2.1 Capacity of the BEC	21
4 Application to Continuous Channels	21
4.1 The Shannon-Hartley Theorem	21
4.2 Implications of Shannon-Hartley Theorem	22
4.2.1 Derivation of Infinite Bandwidth Capacity	22
4.3 Bandwidth-Efficiency Trade-off (C/B vs. E_b/N_0)	22

4.3.1	Derivation of the Shannon Limit	23
Module 3: Huffman Coding		27
1	Huffman Coding	27
1.1	Definition and Concept	27
1.2	The Huffman Coding Algorithm	27
2	Cyclic Codes	29
2.1	Definition of Cyclic Codes	29
2.2	Generator Polynomial $g(x)$	29
2.3	Systematic Encoding of Cyclic Codes	30
2.4	Hardware Encoder using Feedback Shift Registers (FSR)	31
3	Convolutional Codes	31
3.1	Basic Principles and Structure	31
3.2	Representations of Convolutional Codes	32
3.3	Viterbi Decoding Algorithm	32
4	Arithmetic Coding	33
4.1	Core Concept	33
4.2	Encoding Procedure	33

Module 1

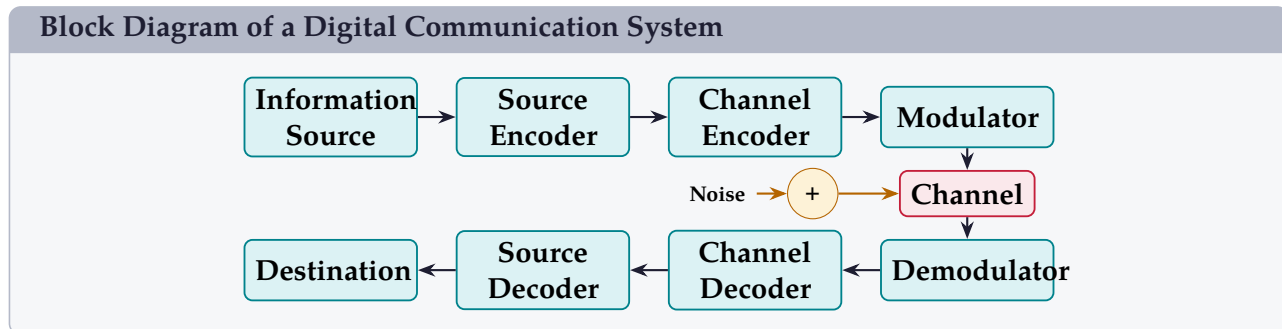
Basics of Information Theory

1. Basics of Information Theory

Information theory is a mathematical framework developed by Claude Shannon in 1948 to quantify, store, and communicate digital information. It defines limits on data compression and transmission rates.

1.1 Model of a Digital Communication System

Before quantifying information, we analyze the basic blocks involved in its transmission:



1.2 Measure of Information Content

The "information" in a message is not related to its semantic meaning, but rather to its **uncertainty** or **surprise value**. An event that is highly likely to happen carries very little information when it occurs, whereas an extremely rare (unlikely) event carries high information.

Information Content: The quantitative measure of the uncertainty associated with the occurrence of an event. For a discrete source emitting a symbol x_i with probability $P(x_i)$, the information content $I(x_i)$ is inversely proportional to $P(x_i)$.

Information Measurement Formula

The information content $I(x_i)$ of symbol x_i is defined as:

$$I(x_i) = \log_r \left(\frac{1}{P(x_i)} \right) = -\log_r P(x_i) \quad (1)$$

The unit of measurement depends on the base r of the logarithm:

Base $r = 2$: Units are **Bits** (Binary digits). $I(x_i) = -\log_2 P(x_i)$.

Base $r = e$: Units are **Nats** (Natural units). $I(x_i) = -\ln P(x_i)$.

Base $r = 10$: Units are **Hartleys** (or Decits). $I(x_i) = -\log_{10} P(x_i)$.

Conversion Relations:

$$1 \text{ Nat} = \log_2 e \approx 1.443 \text{ Bits} \quad (2)$$

$$1 \text{ Hartley} = \log_2 10 \approx 3.322 \text{ Bits} \quad (3)$$

1.3 Entropy for Discrete Ensembles

A discrete information source emits symbols from an alphabet $X = \{x_1, x_2, \dots, x_M\}$ with corresponding probabilities $P = \{P(x_1), P(x_2), \dots, P(x_M)\}$, where $\sum_{i=1}^M P(x_i) = 1$.

Entropy $H(X)$: The average information content per symbol emitted by the source. It represents the mathematical expectation of the information of all symbols, measuring the overall uncertainty of the source.

Entropy Formula

$$H(X) = E[I(x_i)] = \sum_{i=1}^M P(x_i) I(x_i) = -\sum_{i=1}^M P(x_i) \log_2 P(x_i) \quad (\text{bits/symbol}) \quad (4)$$

1.4 Properties of Entropy

Entropy exhibits several fundamental properties that reflect its nature as a measure of uncertainty:

- 1. Non-negativity:** $H(X) \geq 0$. Since $0 \leq P(x_i) \leq 1$, the term $-P(x_i) \log_2 P(x_i)$ is always non-negative.
- 2. Certainty (Minimum Entropy):** $H(X) = 0$ if and only if $P(x_i) = 1$ for some symbol x_i , and $P(x_j) = 0$ for all $j \neq i$. In this case, there is no uncertainty in the source output.
- 3. Maximality (Maximum Entropy):** $H(X) \leq \log_2 M$, where M is the number of symbols in the alphabet. The maximum uncertainty occurs when all symbols are equiprobable, i.e., $P(x_i) = 1/M$ for all i .

1.4.1 Derivation of Maximum Entropy Condition (Proof of Maximality)

To find the probability distribution that maximizes $H(X)$ subject to the constraint $\sum_{i=1}^M P(x_i) = 1$, we use the method of Lagrange multipliers. Let the objective function be:

$$f(P_1, P_2, \dots, P_M) = -\sum_{i=1}^M P_i \ln P_i$$

Subject to the constraint:

$$g(P_1, P_2, \dots, P_M) = \sum_{i=1}^M P_i - 1 = 0$$

Define the Lagrangian function J :

$$J = - \sum_{i=1}^M P_i \ln P_i - \lambda \left(\sum_{i=1}^M P_i - 1 \right)$$

Differentiating J with respect to P_i and setting it to 0 for maximum value:

$$\begin{aligned} \frac{\partial J}{\partial P_i} &= -\ln P_i - 1 - \lambda = 0 \\ \ln P_i &= -(1 + \lambda) \\ P_i &= e^{-(1+\lambda)} \end{aligned}$$

Since the right-hand side is constant and does not depend on the index i , all P_i must be equal:

$$P_1 = P_2 = \dots = P_M = C \quad (\text{a constant})$$

Using the constraint $\sum_{i=1}^M P_i = 1$:

$$M \times C = 1 \implies C = \frac{1}{M} \implies P_i = \frac{1}{M} \quad \text{for all } i.$$

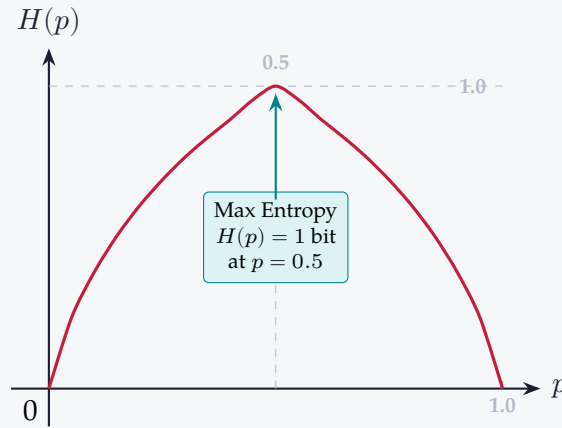
Substituting $P_i = 1/M$ back into the entropy formula yields the maximum entropy:

$$H(X)_{\max} = - \sum_{i=1}^M \frac{1}{M} \log_2 \left(\frac{1}{M} \right) = -M \left(\frac{1}{M} \right) (-\log_2 M) = \log_2 M \quad (\text{Q.E.D.})$$

1.5 Binary Entropy Function

For a binary source with alphabet $X = \{0, 1\}$ and probabilities $P(0) = p$ and $P(1) = 1 - p$, the entropy is denoted by $H(p)$ or $H_b(p)$:

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p) \quad (5)$$

Binary Entropy Function $H(p)$ vs p 

2. Joint, Conditional, and Mutual Information

To analyze communication systems with an input source X and an output destination Y , we define joint and conditional entropy metrics over the joint ensemble (X, Y) .

2.1 Definitions of Entropies

Let $P(x_i)$ be input probabilities, $P(y_j)$ be output probabilities, and $P(x_i, y_j)$ be the joint probability of transmitting x_i and receiving y_j .

Joint and Conditional Entropies

1. **Joint Entropy $H(X, Y)$:** The total average uncertainty of the combined system.

$$H(X, Y) = - \sum_{i=1}^M \sum_{j=1}^N P(x_i, y_j) \log_2 P(x_i, y_j) \quad (6)$$

2. **Conditional Entropy $H(X|Y)$:** The average uncertainty about the input X after the output Y is observed (also called **Equivocation**).

$$H(X|Y) = - \sum_{i=1}^M \sum_{j=1}^N P(x_i, y_j) \log_2 P(x_i|y_j) \quad (7)$$

3. **Conditional Entropy $H(Y|X)$:** The average uncertainty about the output Y when the input X is known (represents the noise introduced by the channel).

$$H(Y|X) = - \sum_{i=1}^M \sum_{j=1}^N P(x_i, y_j) \log_2 P(y_j|x_i) \quad (8)$$

2.1.1 Derivation of Joint Entropy Relations

We derive the relation $H(X, Y) = H(X) + H(Y|X)$ using joint and conditional probability relationships $P(x_i, y_j) = P(x_i)P(y_j|x_i)$:

$$\begin{aligned}
 H(X, Y) &= - \sum_{i=1}^M \sum_{j=1}^N P(x_i, y_j) \log_2 P(x_i, y_j) \\
 &= - \sum_{i=1}^M \sum_{j=1}^N P(x_i, y_j) \log_2 [P(x_i)P(y_j|x_i)] \\
 &= - \sum_{i=1}^M \sum_{j=1}^N P(x_i, y_j) [\log_2 P(x_i) + \log_2 P(y_j|x_i)] \\
 &= - \sum_{i=1}^M \left(\sum_{j=1}^N P(x_i, y_j) \right) \log_2 P(x_i) - \sum_{i=1}^M \sum_{j=1}^N P(x_i, y_j) \log_2 P(y_j|x_i)
 \end{aligned}$$

Using the probability marginalization rule $\sum_{j=1}^N P(x_i, y_j) = P(x_i)$:

$$H(X, Y) = - \sum_{i=1}^M P(x_i) \log_2 P(x_i) + H(Y|X) = H(X) + H(Y|X) \quad (\text{Q.E.D.})$$

Similarly, using $P(x_i, y_j) = P(y_j)P(x_i|y_j)$, we can prove:

$$H(X, Y) = H(Y) + H(X|Y) \quad (9)$$

2.2 Mutual Information

Mutual Information $I(X; Y)$: The amount of information about the source X that is obtained by observing the channel output Y . It measures the reduction in uncertainty of X due to the knowledge of Y .

Mutual Information Formulas

$$I(X; Y) = H(X) - H(X|Y) \quad (\text{bits/symbol}) \quad (10)$$

By symmetry and joint entropy relations, it can also be expressed as:

$$I(X; Y) = H(Y) - H(Y|X) \quad (11)$$

$$I(X; Y) = H(X) + H(Y) - H(X, Y) \quad (12)$$

In terms of probabilities:

$$I(X; Y) = \sum_{i=1}^M \sum_{j=1}^N P(x_i, y_j) \log_2 \left[\frac{P(x_i, y_j)}{P(x_i)P(y_j)} \right] \quad (13)$$

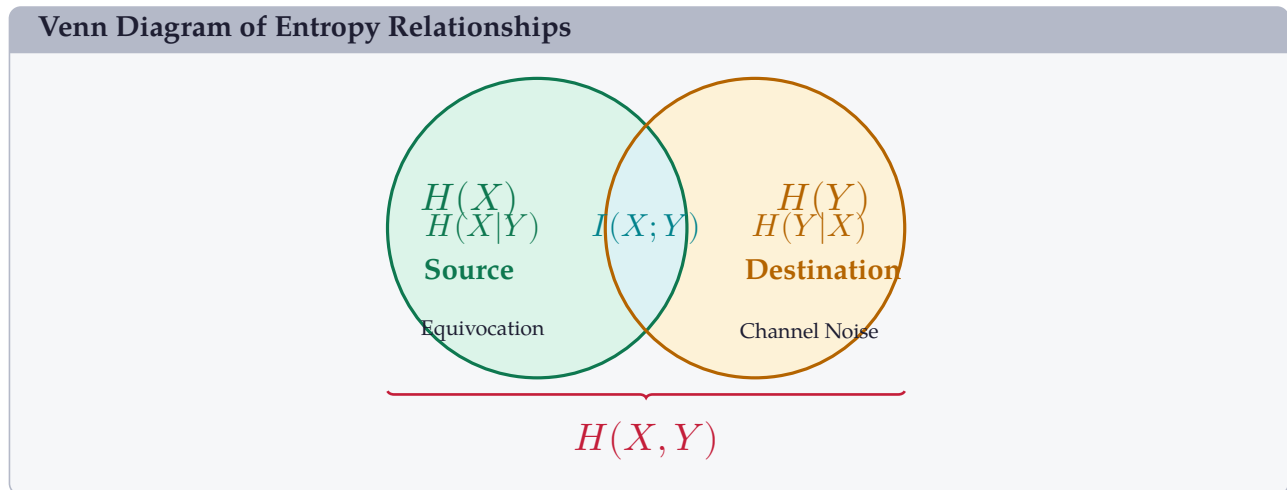
2.3 Properties of Mutual Information

- **Symmetry:** $I(X; Y) = I(Y; X)$. The channel output Y carries the same amount of information about the input X as the input X carries about the output Y .

- **Non-negativity:** $I(X; Y) \geq 0$. The observation of Y can never increase the uncertainty of X on average.
- **Independence Condition:** $I(X; Y) = 0$ if and only if X and Y are statistically independent. In this case, receiving Y provides no information about X .
- **Relation to Channel Capacity:** For deterministic, noiseless channels, conditional uncertainty vanishes, and $I(X; Y) = H(X)$.

2.4 Entropy Relationship Diagram

The relationships between all these entropy measures are summarized in the Venn diagram below:



3. Shannon's First Theorem & Source Coding

Source coding is the process of representing discrete source symbols with binary code words. The goal is to reduce data size by removing redundancy.

3.1 Shannon's Noiseless Coding Theorem

Shannon's First Theorem (Source Coding Theorem): For a discrete memoryless source X with entropy $H(X)$ (bits/symbol), the average codeword length L (bits/symbol) for any uniquely decodable binary code is bounded by:

$$L \geq H(X) \quad (14)$$

By coding sequences of N symbols at a time (block coding), the average code length per source symbol L_N can be made arbitrarily close to $H(X)$ as $N \rightarrow \infty$:

$$\lim_{N \rightarrow \infty} L_N = H(X) \quad (15)$$

3.2 Performance Metrics of Source Codes

For a source emitting symbols s_i with probability $P(s_i)$ and assigned codeword length l_i (in bits):

Average Length, Efficiency, and Redundancy

1. Average Codeword Length (L):

$$L = \sum_{i=1}^M P(s_i) l_i \quad (\text{bits/symbol}) \quad (16)$$

2. Coding Efficiency (η):

$$\eta = \frac{H(X)}{L} \times 100\% \quad (17)$$

3. Code Redundancy (R):

$$R = 1 - \eta = 1 - \frac{H(X)}{L} \quad (18)$$

3.3 Prefix Codes and Kraft-McMillan Inequality

A code is **uniquely decodable** if any concatenated sequence of codewords can be decoded in only one way.

Prefix Code (Instantaneous Code): A uniquely decodable code with the property that no codeword is a prefix of any other codeword. This allows a sequence to be decoded symbol-by-symbol instantaneously as it is received, without waiting for the end of the sequence.

Kraft-McMillan Inequality

A prefix code (or any uniquely decodable code) with alphabet size D (usually binary, $D = 2$) and codeword lengths $l_1, l_2, \dots, l_M$ exists if and only if:

$$\sum_{i=1}^M D^{-l_i} \leq 1 \quad (19)$$

For binary codes, this is:

$$\sum_{i=1}^M 2^{-l_i} \leq 1 \quad (20)$$

If the inequality is satisfied with equality ($\sum 2^{-l_i} = 1$), the code is called a **complete code** (no further codewords can be added without violating the prefix property).

3.4 Shannon-Fano Coding Algorithm

Shannon-Fano coding is a top-down algorithm to construct prefix codes. The procedure is:

Step 1: Sort the source symbols in descending order of their probabilities.

Step 2: Divide the sorted list of symbols into two subsets such that the sum of probabilities of the two subsets is as close to equal as possible.

Step 3: Assign binary code '0' to all symbols in the first (upper) subset and '1' to all symbols in the second (lower) subset.

Step 4: Recursively apply Steps 2 and 3 to each subset until each sub-sublist contains exactly one symbol.

Step 5: The codeword for each symbol is the sequence of 0s and 1s assigned to it from the root of divisions.

Worked Example: Shannon-Fano Coding

Problem: A discrete memoryless source emits 5 symbols $\{s_1, s_2, s_3, s_4, s_5\}$ with probabilities $P = \{0.35, 0.30, 0.20, 0.10, 0.05\}$.

- Construct a binary Shannon-Fano code.
- Calculate the entropy $H(X)$ of the source.
- Calculate the average codeword length L and the efficiency η .

Solution:

(a) Code Construction Table:

Symbol	Prob.	Stage 1	Stage 2	Stage 3	Codeword	Length (l_i)
s_1	0.35	0	0		00	2
s_2	0.30	0	1		01	2
s_3	0.20	1	0		10	2
s_4	0.10	1	1	0	110	3
s_5	0.05	1	1	1	111	3

Division Rationale:

- Stage 1:** Split between $\{s_1, s_2\}$ (sum = 0.65) and $\{s_3, s_4, s_5\}$ (sum = 0.35). Difference is 0.30. (Splitting between s_1 and others yields difference $0.35 - 0.65 = 0.30$, so both partitions are valid. Let's use the first).
- Stage 2:** Split upper subset $\{s_1\}$ (0.35) and $\{s_2\}$ (0.30). Split lower subset $\{s_3\}$ (0.20) and $\{s_4, s_5\}$ (0.15).
- Stage 3:** Split $\{s_4\}$ (0.10) and $\{s_5\}$ (0.05).

(b) Source Entropy $H(X)$:

$$\begin{aligned}
H(X) &= - \sum_{i=1}^5 P(s_i) \log_2 P(s_i) \\
&= -[0.35 \log_2 0.35 + 0.30 \log_2 0.30 + 0.20 \log_2 0.20 + 0.10 \log_2 0.10 + 0.05 \log_2 0.05] \\
&= -[0.35(-1.5146) + 0.30(-1.7370) + 0.20(-2.3219) + 0.10(-3.3219) + 0.05(-4.3219)] \\
&= -[-0.5301 - 0.5211 - 0.4644 - 0.3322 - 0.2161] = 2.0639 \text{ bits/symbol}
\end{aligned}$$

(c) Average Codeword Length L and Efficiency η :

$$\begin{aligned}
L &= \sum_{i=1}^5 P(s_i) l_i = (0.35 \times 2) + (0.30 \times 2) + (0.20 \times 2) + (0.10 \times 3) + (0.05 \times 3) \\
&= 0.70 + 0.60 + 0.40 + 0.30 + 0.15 = 2.15 \text{ bits/symbol}
\end{aligned}$$

$$\eta = \frac{H(X)}{L} \times 100\% = \frac{2.0639}{2.15} \times 100\% \approx 95.99\%$$

$$\text{Redundancy } R = 1 - 0.9599 = 0.0401 \text{ or } 4.01\%$$

Kraft Inequality Verification:

$$\sum_{i=1}^5 2^{-l_i} = 2^{-2} + 2^{-2} + 2^{-2} + 2^{-3} + 2^{-3} = 0.25 + 0.25 + 0.25 + 0.125 + 0.125 = 1.0 \leq 1$$

The sum equals 1.0. This indicates that the code is prefix-free and is a complete code.

Quick Revision — Module I

Basics & Source Coding | PE-EC603D

Key Formulas at a Glance

Self-Information	$I(x_i) = \log_2 \left(\frac{1}{P(x_i)} \right) = -\log_2 P(x_i)$
Entropy (Average Info)	$H(X) = -\sum_{i=1}^M P(x_i) \log_2 P(x_i)$
Joint Entropy	$H(X, Y) = -\sum \sum P(x_i, y_j) \log_2 P(x_i, y_j)$
Conditional Entropies	$H(X Y) = H(X, Y) - H(Y) \quad \text{and} \quad H(Y X) = H(X, Y) - H(X)$
Mutual Information	$I(X; Y) = H(X) - H(X Y) = H(Y) - H(Y X)$
Average Code Length	$L = \sum P(s_i) l_i$
Coding Efficiency	$\eta = \frac{H(X)}{L} \times 100\%$
Kraft Inequality	$\sum 2^{-l_i} \leq 1 \quad (\text{For binary prefix codes})$

One-Line Definitions

- **Information Content:** A measure of surprise or uncertainty of an event, dependent only on its probability.
- **Entropy:** The average uncertainty or information content per symbol emitted by a source.
- **Equivocation $H(X|Y)$:** The residual uncertainty of the input X after output Y is observed at the receiver.
- **Mutual Information:** The amount of information shared between the transmitter and receiver.
- **Prefix Code:** A code in which no codeword is a prefix of another, enabling instantaneous decoding.
- **Kraft Inequality:** A mathematical condition determining the existence of prefix codes for given lengths.
- **Source Coding Theorem:** Shannon's limit stating that average codeword length cannot be less than source entropy.
- **Redundancy:** The percentage of code space wasted by transmitting non-informative bits ($R = 1 - \eta$).

Frequently Asked / Viva Questions

Q1. State the units of information and their conversion relations.

Bits (base 2), Nats (base e), and Hartleys (base 10). Conversions: 1 Nat =

Frequently Asked / Viva Questions (contd.)

1.443 Bits; 1 Hartley = 3.322 Bits.

Q2. Under what condition is the entropy of a source maximized?

When all M symbols of the source alphabet are equiprobable, i.e., $P(x_i) = 1/M$ for all i . The maximum entropy is $H(X)_{\max} = \log_2 M$.

Q3. What is the physical significance of conditional entropy $H(Y|X)$?

It represents the average uncertainty of the output Y when the input X is known. Physically, it measures the noise added by the channel.

Q4. Explain the term "Equivocation" in communication channels.

Equivocation is $H(X|Y)$, which is the average uncertainty about the transmitted symbol X after receiving symbol Y . It represents lost information.

Q5. Why is Mutual Information symmetric?

$I(X;Y) = I(Y;X) = H(X) + H(Y) - H(X,Y)$. It represents the shared information between X and Y , making the direction of analysis irrelevant.

Q6. State Shannon's Source Coding Theorem.

The average length L of any uniquely decodable binary code for a source X satisfies $L \geq H(X)$. The minimum average code length is limited by the source entropy.

Q7. What is an instantaneous code (prefix code)?

A code where no codeword is a prefix of any other codeword. It is decodable immediately upon receiving the last bit of the codeword, without looking ahead.

Q8. What is the significance of the Kraft-McMillan Inequality?

It provides a necessary and sufficient condition for the existence of an instantaneous (prefix) code for a given set of codeword lengths.

Q9. If a source has an entropy of 1.8 bits/symbol and is encoded using a code with average length 2 bits/symbol, find efficiency and redundancy.

Efficiency $\eta = \frac{1.8}{2} \times 100 = 90\%$. Redundancy $R = 100\% - 90\% = 10\%$.

Q10. Why does block coding increase source coding efficiency?

Block coding groups N symbols together, allowing the average length per symbol to approach the entropy $H(X)$ more closely as $N \rightarrow \infty$ by utilizing joint probabilities.

Q11. Can the efficiency of a source code be greater than 100%?

No. According to Shannon's Source Coding Theorem, $L \geq H(X)$, meaning efficiency $\eta = H(X)/L \leq 1.0$ (or 100%).

Q12. State the main difference between Shannon-Fano and Huffman coding approaches.

Frequently Asked / Viva Questions (contd.)

Shannon-Fano is a top-down approach that partitions probabilities into subgroups, while Huffman coding is a bottom-up approach that recursively merges the lowest probabilities.

Comparison of Source Coding Techniques

Parameter	Fixed-Length Coding	Variable-Length Coding
Codeword Length	Equal length ($l_i = \lceil \log_2 M \rceil$) for all symbols.	Different lengths based on symbol probabilities.
Probability Sensitivity	Ignores probability distribution.	Allocates shorter codes to high-probability symbols.
Efficiency	Low if symbols have unequal probabilities.	High (approaches Shannon's limit $H(X)$).
Synchronization	Easy; boundaries are known.	Harder; requires prefix-free property (Prefix codes).
Examples	ASCII, Standard PCM.	Shannon-Fano, Huffman coding.

Module 2

Markov Sources

1. Markov Sources

Information sources are categorized based on whether current outputs depend on past outputs. A source is **memoryless** if symbols are emitted independently. A source has **memory** if current output probabilities are conditioned on previous symbols.

1.1 Definition of Markov Sources

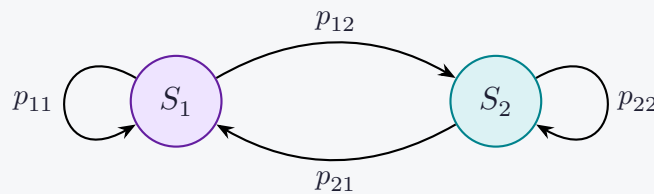
Markov Source: A discrete source with memory where the probability of emitting the next symbol depends on a finite number of preceding symbols.

In a **first-order Markov source**, the probability of the next symbol depends only on the immediately preceding symbol:

$$P(S_t = s_j \mid S_{t-1} = s_i, S_{t-2} = s_k, \dots) = P(S_t = s_j \mid S_{t-1} = s_i) = p_{ij} \quad (21)$$

Where p_{ij} is the transition probability from state s_i to state s_j .

State Transition Diagram of a 2-State Markov Source



1.2 Transition Probability Matrix (TPM)

The transition probabilities of a Markov chain with K states are organized into a square matrix P :

$$P = \begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1K} \\ p_{21} & p_{22} & \cdots & p_{2K} \\ \vdots & \vdots & \ddots & \vdots \\ p_{K1} & p_{K2} & \cdots & p_{KK} \end{bmatrix} \quad (22)$$

Since the source must transition to one of the possible states, the sum of probabilities in each row is exactly 1:

$$\sum_{j=1}^K p_{ij} = 1 \quad \text{for all } i = 1, 2, \dots, K \quad (23)$$

1.3 Stationary State Probabilities

Let $\mathbf{W} = [w_1, w_2, \dots, w_K]$ represent the stationary (steady-state) probability vector, where w_i is the probability that the source is in state i in the long run.

Stationary State Probability Equations

The stationary probability vector satisfies:

$$\mathbf{W}P = \mathbf{W} \quad \text{and} \quad \sum_{i=1}^K w_i = 1 \quad (24)$$

Which yields a system of linear equations:

$$w_1 p_{11} + w_2 p_{21} + \dots + w_K p_{K1} = w_1 \quad (25)$$

$$w_1 p_{12} + w_2 p_{22} + \dots + w_K p_{K2} = w_2 \quad (26)$$

$$\vdots \quad (27)$$

$$w_1 + w_2 + \dots + w_K = 1 \quad (28)$$

1.4 Entropy of Markov Sources (Entropy Rate)

Because the output probability depends on the current state, we first calculate the entropy of each state individually, and then compute the weighted average over all states.

Markov Source Entropy Rate Formulas

- Entropy of State i (H_i):** The uncertainty of the next symbol given the source is currently in state i .

$$H_i = - \sum_{j=1}^K p_{ij} \log_2 p_{ij} \quad (\text{bits/symbol}) \quad (29)$$

- Average Entropy of Markov Source ($H(X)$):** The long-run average uncertainty per symbol.

$$H(X) = \sum_{i=1}^K w_i H_i = - \sum_{i=1}^K w_i \sum_{j=1}^K p_{ij} \log_2 p_{ij} \quad (\text{bits/symbol}) \quad (30)$$

Worked Example: Entropy Rate of a Markov Source

Problem: A 2-state Markov source has the following transition probability matrix:

$$P = \begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{bmatrix}$$

- Find the stationary state probabilities w_1 and w_2 .
- Calculate the state entropies H_1 and H_2 .
- Find the average entropy rate $H(X)$ of this source.

Solution:

(a) Find stationary probabilities w_1 and w_2 : Using $\mathbf{W}P = \mathbf{W}$:

$$[w_1, w_2] \begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{bmatrix} = [w_1, w_2]$$

This yields the equations:

$$\begin{aligned} 0.8w_1 + 0.3w_2 &= w_1 \implies 0.3w_2 = 0.2w_1 \implies w_2 = \frac{2}{3}w_1 \\ 0.2w_1 + 0.7w_2 &= w_2 \implies 0.2w_1 = 0.3w_2 \quad (\text{redundant}) \end{aligned}$$

Using the probability normalization constraint $w_1 + w_2 = 1$:

$$w_1 + \frac{2}{3}w_1 = 1 \implies \frac{5}{3}w_1 = 1 \implies w_1 = 0.6 \quad \text{and} \quad w_2 = 0.4$$

(b) Calculate state entropies H_1 and H_2 :

$$\begin{aligned} H_1 &= -[0.8 \log_2 0.8 + 0.2 \log_2 0.2] = -[0.8(-0.3219) + 0.2(-2.3219)] = 0.7219 \text{ bits/symbol} \\ H_2 &= -[0.3 \log_2 0.3 + 0.7 \log_2 0.7] = -[0.3(-1.7370) + 0.7(-0.5146)] = 0.8813 \text{ bits/symbol} \end{aligned}$$

(c) Find the average entropy rate $H(X)$:

$$\begin{aligned} H(X) &= w_1 H_1 + w_2 H_2 \\ &= (0.6 \times 0.7219) + (0.4 \times 0.8813) \\ &= 0.4331 + 0.3525 = 0.7856 \text{ bits/symbol} \end{aligned}$$

Answer: $w_1 = 0.6, w_2 = 0.4$; $H_1 = 0.7219, H_2 = 0.8813$ bits/sym; average entropy is **0.7856 bits/symbol**.

2. Shannon's Second Theorem (Noisy Channel Coding Theorem)

When transmitting data over physical media, noise corrupts signals, causing bit errors. Shannon proved that error-free communication is possible even over noisy channels, provided the transmission rate does not exceed a threshold called **Channel Capacity**.

2.1 Channel Capacity Concept

Channel Capacity (C): The maximum rate at which information can be transmitted reliably through a communication channel. It is mathematically defined as the maximum of the mutual information $I(X;Y)$ over all possible input probability distributions $P(x)$:

$$C = \max_{P(x)} I(X;Y) \quad (\text{bits/channel-use}) \quad (31)$$

2.2 Noisy Channel Coding Theorem

Shannon's Second Theorem (Noisy Channel Coding Theorem)

Let a discrete memoryless channel have capacity C .

- Theorem:** If the source transmission rate R (bits/second) is less than or equal to C ($R \leq C$), there exists a coding scheme (with block length N sufficiently large) such that the probability of error at the receiver, P_e , can be made arbitrarily small:

$$\lim_{N \rightarrow \infty} P_e = 0 \quad (32)$$

- Converse:** If $R > C$, it is impossible to design a coding scheme that achieves error-free transmission. The probability of bit error P_e is bounded away from zero, and reliable communication cannot occur.

This theorem is historically significant because it proved that channel noise limits the **rate of transmission**, not its **accuracy**. We do not need noiseless channels for error-free data transfer; we only need appropriate error-correcting codes.

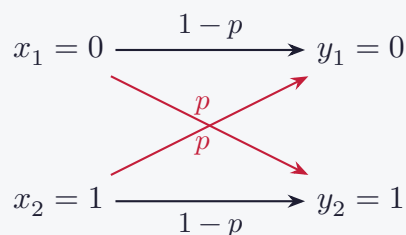
3. Calculation of Channel Capacity for Discrete Channels

A discrete channel is represented by input alphabet X , output alphabet Y , and a channel transition matrix containing conditional probabilities $P(y_j|x_i)$.

3.1 Binary Symmetric Channel (BSC)

The BSC is the most common model for binary transmission. The input and output alphabets are binary $\{0, 1\}$. The probability of a bit being transmitted correctly is $1 - p$, and the probability of a bit being flipped (error) is p .

Binary Symmetric Channel (BSC) Transition Model



3.1.1 Derivation of BSC Channel Capacity

We calculate capacity using $I(X; Y) = H(Y) - H(Y|X)$. First, evaluate the conditional entropy $H(Y|X)$:

$$\begin{aligned}
 H(Y|X) &= - \sum_{i=1}^2 P(x_i) \sum_{j=1}^2 P(y_j|x_i) \log_2 P(y_j|x_i) \\
 &= -P(x_1) [P(y_1|x_1) \log_2 P(y_1|x_1) + P(y_2|x_1) \log_2 P(y_2|x_1)] \\
 &\quad - P(x_2) [P(y_1|x_2) \log_2 P(y_1|x_2) + P(y_2|x_2) \log_2 P(y_2|x_2)] \\
 &= -P(x_1) [(1-p) \log_2(1-p) + p \log_2 p] - P(x_2) [p \log_2 p + (1-p) \log_2(1-p)]
 \end{aligned}$$

Since $P(x_1) + P(x_2) = 1$:

$$H(Y|X) = -p \log_2 p - (1-p) \log_2(1-p) = H(p) \quad (\text{independent of input distribution } P(x))$$

Substitute this into the capacity equation:

$$C = \max_{P(x)} [H(Y) - H(Y|X)] = \max_{P(x)} H(Y) - H(p)$$

Since Y is a binary variable, its entropy $H(Y)$ is maximized when Y has equiprobable outputs, i.e., $P(y_1) = P(y_2) = 0.5$, which yields $H(Y)_{\max} = 1$. This maximum is achieved when the input symbols are equiprobable ($P(x_1) = P(x_2) = 0.5$).

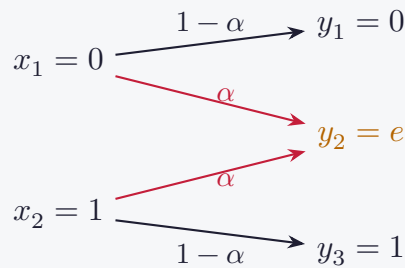
BSC Channel Capacity

$$C_{\text{BSC}} = 1 - H(p) = 1 + p \log_2 p + (1-p) \log_2(1-p) \quad (\text{bits/channel-use}) \quad (33)$$

3.2 Binary Erasure Channel (BEC)

In a BEC, the transmitter sends binary digits $\{0, 1\}$. Due to noise, the receiver may fail to decode the bit. Rather than receiving an incorrect bit, the receiver recognizes a lost bit and labels it as an erasure symbol ' e '. The probability of erasure is α .

Binary Erasure Channel (BEC) Transition Model



3.2.1 Capacity of the BEC

Let the input probability be $P(x = 0) = \beta$, and $P(x = 1) = 1 - \beta$. The output probabilities are:

$$\begin{aligned} P(y = 0) &= (1 - \alpha)\beta \\ P(y = e) &= \alpha\beta + \alpha(1 - \beta) = \alpha \\ P(y = 1) &= (1 - \alpha)(1 - \beta) \end{aligned}$$

To find capacity, we compute conditional entropy $H(Y|X)$: Since for any input x , the output is correct with probability $1 - \alpha$ and erased with probability α :

$$H(Y|X) = H(\alpha) = -\alpha \log_2 \alpha - (1 - \alpha) \log_2 (1 - \alpha)$$

The output entropy $H(Y)$ is:

$$\begin{aligned} H(Y) &= -P(y = 0) \log_2 P(y = 0) - P(y = e) \log_2 P(y = e) - P(y = 1) \log_2 P(y = 1) \\ &= -(1 - \alpha)\beta \log_2 [(1 - \alpha)\beta] - \alpha \log_2 \alpha - (1 - \alpha)(1 - \beta) \log_2 [(1 - \alpha)(1 - \beta)] \\ &= -\alpha \log_2 \alpha - (1 - \alpha) \log_2 (1 - \alpha) + (1 - \alpha) [-\beta \log_2 \beta - (1 - \beta) \log_2 (1 - \beta)] \\ &= H(\alpha) + (1 - \alpha)H(X) \end{aligned}$$

Using $I(X; Y) = H(Y) - H(Y|X)$:

$$I(X; Y) = H(\alpha) + (1 - \alpha)H(X) - H(\alpha) = (1 - \alpha)H(X)$$

To maximize $I(X; Y)$, we maximize $H(X)$. Since X is binary, its maximum entropy is $H(X)_{\max} = 1$ bit (at $\beta = 0.5$).

BEC Channel Capacity

$$C_{\text{BEC}} = 1 - \alpha \quad (\text{bits/channel-use}) \quad (34)$$

This shows that erasures reduce capacity linearly: if 20% of bits are erased ($\alpha = 0.2$), the capacity is 80% of a noiseless channel ($C = 0.8$ bits/use).

4. Application to Continuous Channels

Discrete channel models fail when signals are continuous voltage waveforms corrupted by additive white Gaussian noise (AWGN). We analyze these systems using continuous capacities.

4.1 The Shannon-Hartley Theorem

Shannon-Hartley Theorem: The maximum rate of transmission over a continuous channel of bandwidth B (Hz), corrupted by Additive White Gaussian Noise (AWGN) with signal power S and noise power N , is:

$$C = B \log_2 \left(1 + \frac{S}{N} \right) \quad (\text{bits/second}) \quad (35)$$

If noise spectral density is $N_0/2$ (double-sided) or N_0 (single-sided), the total noise power

is $N = N_0 B$:

$$C = B \log_2 \left(1 + \frac{S}{N_0 B} \right) \quad (36)$$

4.2 Implications of Shannon-Hartley Theorem

This relation highlights several trade-offs:

- a. **Signal-to-Noise Ratio (SNR) vs Bandwidth (B):** We can exchange bandwidth for SNR. A system with small bandwidth can achieve high capacity if the SNR is high, and vice-versa.
- b. **Infinite Bandwidth capacity limit:** If we keep signal power S and noise density N_0 constant, and let the bandwidth B grow to infinity, the capacity does not grow to infinity. Instead, it reaches a thermodynamic limit:

4.2.1 Derivation of Infinite Bandwidth Capacity

We evaluate the limit of C as $B \rightarrow \infty$:

$$\lim_{B \rightarrow \infty} C = \lim_{B \rightarrow \infty} B \log_2 \left(1 + \frac{S}{N_0 B} \right)$$

Let $x = \frac{S}{N_0 B}$. As $B \rightarrow \infty$, $x \rightarrow 0$. We express B as $B = \frac{S}{N_0 x}$:

$$\begin{aligned} \lim_{B \rightarrow \infty} C &= \lim_{x \rightarrow 0} \frac{S}{N_0 x} \log_2(1 + x) \\ &= \frac{S}{N_0} \lim_{x \rightarrow 0} \log_2(1 + x)^{1/x} \end{aligned}$$

By the definition of the natural base e , $\lim_{x \rightarrow 0} (1 + x)^{1/x} = e$:

$$\lim_{B \rightarrow \infty} C = \frac{S}{N_0} \log_2 e \approx 1.4427 \frac{S}{N_0}$$

Infinite Bandwidth Capacity Limit

$$C_\infty = 1.44 \frac{S}{N_0} \quad (\text{bits/second}) \quad (37)$$

This represents the absolute physical limit on transmission capacity under a power constraint, even with infinite bandwidth.

4.3 Bandwidth-Efficiency Trade-off (C/B vs. E_b/N_0)

We define the **spectral efficiency** as C/B (bps/Hz). Let E_b be the energy per bit transmitted. Since signal power is energy per unit time:

$$S = C \times E_b$$

Substitute this into the Shannon-Hartley equation:

$$\begin{aligned} \frac{C}{B} &= \log_2 \left(1 + \frac{C E_b}{B N_0} \right) \\ 2^{C/B} &= 1 + \frac{C}{B} \left(\frac{E_b}{N_0} \right) \end{aligned}$$

Solving for E_b/N_0 :

Bandwidth-Efficiency Relation

$$\frac{E_b}{N_0} = \frac{2^{C/B} - 1}{C/B} \quad (38)$$

4.3.1 Derivation of the Shannon Limit

To find the absolute minimum value of E_b/N_0 required for error-free communication, we take the limit as the spectral efficiency $C/B \rightarrow 0$ (highly redundant coding over very wide bandwidth):

$$\lim_{C/B \rightarrow 0} \frac{E_b}{N_0} = \lim_{y \rightarrow 0} \frac{2^y - 1}{y}$$

Using L'Hôpital's rule (differentiating numerator and denominator with respect to y):

$$\lim_{y \rightarrow 0} \frac{\ln 2 \cdot 2^y}{1} = \ln 2 \approx 0.693$$

Expressed in decibels (dB):

$$\left(\frac{E_b}{N_0} \right)_{\text{dB}} = 10 \log_{10}(0.693) \approx -1.59 \text{ dB}$$

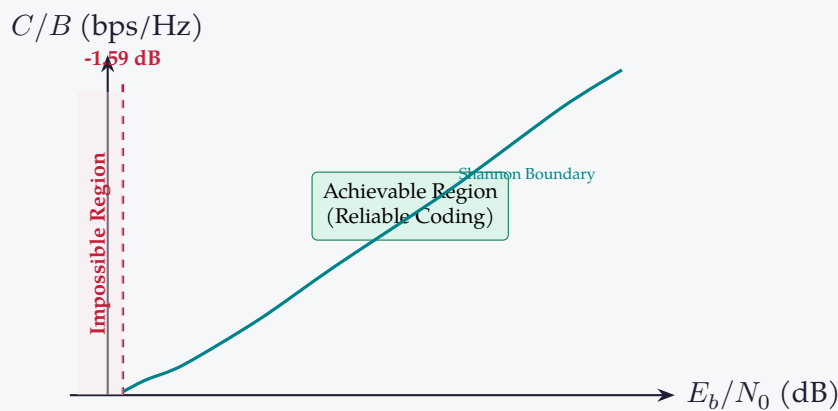
The Shannon Limit

The minimum energy-to-noise ratio required for reliable transmission is:

$$\left(\frac{E_b}{N_0} \right)_{\min} = -1.59 \text{ dB} \quad (39)$$

If a system operates below -1.59 dB , error-free communication is physically impossible.

Spectral Efficiency C/B vs E_b/N_0 (Shannon Boundary)



Quick Revision — Module II

Markov Sources & Channel Capacity | PE-EC603D

Key Formulas at a Glance

Markov Transition Prob.	$\sum_{j=1}^K p_{ij} = 1$ (Row sum of TPM equals 1)
Steady-State Vector	$\mathbf{W}P = \mathbf{W}$ and $\sum w_i = 1$
State Entropy	$H_i = -\sum_{j=1}^K p_{ij} \log_2 p_{ij}$
Markov Source Entropy	$H(X) = \sum w_i H_i$
Channel Capacity	$C = \max_{P(x)} I(X; Y)$ (bits/channel-use)
BSC Capacity	$C_{\text{BSC}} = 1 - H(p) = 1 + p \log_2 p + (1 - p) \log_2 (1 - p)$
BEC Capacity	$C_{\text{BEC}} = 1 - \alpha$
Shannon-Hartley Theorem	$C = B \log_2 \left(1 + \frac{S}{N}\right) = B \log_2 \left(1 + \frac{S}{N_0 B}\right)$ (bps)
Infinite Bandwidth Limit	$C_\infty = 1.44 \frac{S}{N_0}$
Shannon Limit (Min SNR)	$\left(\frac{E_b}{N_0}\right)_{\min} = \ln 2 \approx 0.693 \approx -1.59 \text{ dB}$

One-Line Definitions

- **Markov Source:** A source with memory where current output statistics depend on previous state history.
- **Transition Matrix (TPM):** A matrix storing conditional probabilities of moving between Markov states.
- **Steady-State Vector:** Stationary probabilities indicating long-run occupancy rates of Markov states.
- **Channel Capacity:** The maximum rate of information transmission that can be achieved over a channel.
- **Binary Symmetric Channel:** A binary channel model with symmetric error (crossover) probability p .
- **Binary Erasure Channel:** A channel model where bits are either received correctly or recognized as lost (erased).
- **AWGN Channel:** A continuous channel model corrupted by additive white Gaussian noise.
- **Shannon Limit:** The absolute minimum value of E_b/N_0 (-1.59 dB) required for error-free transmission.

Frequently Asked / Viva Questions

Q1. State the main condition that defines a first-order Markov source.

The output probability at any time instant depends only on the output symbol of the immediately preceding time instant.

Q2. Why does the sum of transition probabilities in each row of a TPM equal 1?

A row represents all transition possibilities leaving a specific state. The source must transition to one of the possible states, so the sum of these probabilities is certain (1).

Q3. Explain the physical meaning of Shannon's Second Theorem.

If transmission rate $R \leq C$, error-free communication is possible using coding. If $R > C$, it is physically impossible to achieve error-free communication.

Q4. What happens to the capacity of a BSC when the crossover probability $p = 0.5$?

Capacity $C = 1 - H(0.5) = 1 - 1 = 0$. The channel output is completely random and independent of input, meaning no information can be transmitted.

Q5. What is an "erasure" symbol in the context of a BEC?

It is a symbol received as a lost bit (labeled 'e'), meaning the receiver knows a bit was sent but cannot determine if it was '0' or '1'.

Q6. What are the components of noise in the Shannon-Hartley theorem?

Additive (noise is added to signal), White (flat power spectral density across all frequencies), and Gaussian (probability density function of noise voltage is Gaussian).

Q7. Why can't we achieve infinite channel capacity by making the bandwidth infinite?

As bandwidth increases, the noise power $N = N_0 B$ also increases. The capacity reaches a thermodynamic limit of $1.44S/N_0$ bps.

Q8. Explain the bandwidth-SNR trade-off using a practical example.

In telephone channels with small bandwidth, high SNR is used to achieve high rates. In satellite links with low power (low SNR), wide bandwidths are used.

Q9. State the value of the Shannon Limit and its practical significance.

-1.59 dB. It is the absolute minimum signal-to-noise ratio per bit required to transmit information over an AWGN channel without errors.

Q10. Under what conditions does the capacity of a channel equal its bandwidth?

When the signal-to-noise ratio $S/N = 1$ (or 0 dB), because $C = B \log_2(1 + 1) = B \log_2(2) = B$ bps.

Q11. Distinguish between a lossless channel and a noiseless channel.

In a lossless channel, $H(Y|X) = 0$ (no noise, inputs map uniquely to outputs). In a noiseless channel, $H(X|Y) = 0$ (outputs map uniquely to inputs, no ambiguity).

Frequently Asked / Viva Questions (contd.)

Q12. How do you calculate the entropy of a Markov source?

First, find steady-state probabilities $\mathbf{W}$. Then, calculate state entropies H_i . The average source entropy is the weighted sum $H(X) = \sum w_i H_i$.

Comparison of BSC and BEC Models

Parameter	Binary Symmetric Channel (BSC)	Binary Erasure Channel (BEC)
Input Alphabet	Binary: $\{0, 1\}$	Binary: $\{0, 1\}$
Output Alphabet	Binary: $\{0, 1\}$	Ternary: $\{0, e, 1\}$ (where e is erasure)
Error Type	Bit flip (crossover), $0 \leftrightarrow 1$.	Bit loss, input becomes unreadable symbol e .
Error Knowledge	Receiver does not know which bits are flipped.	Receiver knows exactly which bits are erased.
Capacity Formula	$C = 1 - H(p)$ bits/use	$C = 1 - \alpha$ bits/use

Module 3

Huffman Coding

1. Huffman Coding

Huffman coding is an optimal source coding technique used for lossy/lossless data compression. It belongs to the class of prefix-free codes, which are uniquely detectable.

1.1 Definition and Concept

Huffman Coding: A bottom-up, variable-length source coding algorithm that assigns binary codes to symbols based on their probabilities. High-probability symbols are assigned shorter codewords, and low-probability symbols are assigned longer codewords, minimizing average codeword length.

1.2 The Huffman Coding Algorithm

- Step 1:** Sort the source symbols in descending order of their probabilities.
- Step 2:** Combine the two lowest-probability symbols into a single node (parent node) with a probability equal to the sum of their probabilities.
- Step 3:** Assign binary '0' to one branch of the pair (usually the upper/higher probability branch) and '1' to the other branch.
- Step 4:** Place the combined node back into the list and re-sort all items in descending order.
- Step 5:** Repeat Steps 2 to 4 recursively until only one root node (probability = 1.0) remains.
- Step 6:** The codeword for each symbol is read by tracing the path from the root of the tree down to the symbol leaf node.

Worked Example: Huffman Coding

Problem: A discrete source emits 5 symbols $\{s_1, s_2, s_3, s_4, s_5\}$ with probabilities $P = \{0.4, 0.2, 0.2, 0.1, 0.1\}$.

- a. Construct the Huffman code tree and determine the codewords.
- b. Calculate the average codeword length L and coding efficiency η .

Solution:

(a) Tree Construction and Codewords: Tracing from the lowest probabilities up (bottom-up merging):

- Merge s_4 (0.1) and s_5 (0.1) to create node N_1 with probability 0.2. Assign 0 to s_4 and 1 to s_5 .
- We now have nodes with probabilities $\{0.4, 0.2, 0.2, 0.2\}$. Merge the two lowest, say s_3 (0.2) and N_1 (0.2) to create N_2 with probability 0.4. Assign 0 to s_3 and 1 to N_1 .
- We now have $\{0.4, 0.4, 0.2\}$. Merge s_2 (0.2) and N_2 (0.4) to create N_3 with probability 0.6. Assign 0 to s_2 and 1 to N_2 .
- Merge s_1 (0.4) and N_3 (0.6) to create the root (1.0). Assign 0 to s_1 and 1 to N_3 .

Symbol	Probability (P_i)	Codeword Path	Length (l_i)
s_1	0.4	0	1
s_2	0.2	10	2
s_3	0.2	110	3
s_4	0.1	1110	4
s_5	0.1	1111	4

(b) Calculations: Source Entropy $H(X)$:

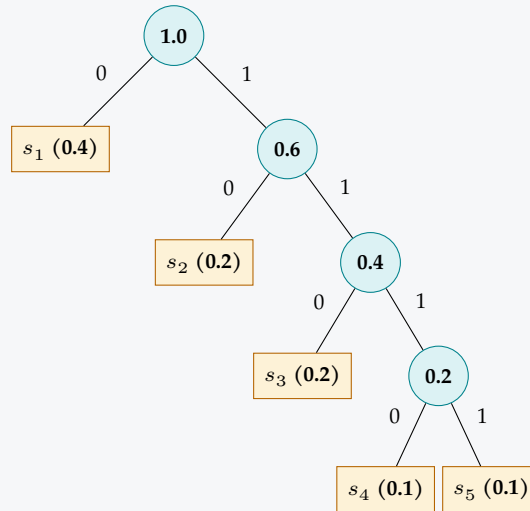
$$\begin{aligned}
 H(X) &= -[0.4 \log_2 0.4 + 2 \times (0.2 \log_2 0.2) + 2 \times (0.1 \log_2 0.1)] \\
 &= -[0.4(-1.3219) + 0.4(-2.3219) + 0.2(-3.3219)] \\
 &= -[-0.5288 - 0.9288 - 0.6644] = 2.122 \text{ bits/symbol}
 \end{aligned}$$

Average Codeword Length L :

$$\begin{aligned}
 L &= \sum P_i l_i = (0.4 \times 1) + (0.2 \times 2) + (0.2 \times 3) + (0.1 \times 4) + (0.1 \times 4) \\
 &= 0.4 + 0.4 + 0.6 + 0.4 + 0.4 = 2.2 \text{ bits/symbol}
 \end{aligned}$$

$$\text{Efficiency } \eta = \frac{H(X)}{L} = \frac{2.122}{2.2} \times 100\% = 96.45\%$$

Huffman Coding Tree Diagram



2. Cyclic Codes

Channel coding techniques inject structured redundancy into data packets to allow detection and correction of channel-induced errors.

2.1 Definition of Cyclic Codes

Cyclic Code: A linear block code (n, k) with the algebraic property that any cyclic shift of a valid codeword is also a valid codeword.

If $C = (c_0, c_1, \dots, c_{n-1})$ is a codeword, then $C^{(1)} = (c_{n-1}, c_0, \dots, c_{n-2})$ is also a codeword.

In polynomial representation, a codeword is represented as:

$$c(x) = c_0 + c_1x + c_2x^2 + \dots + c_{n-1}x^{n-1} \quad (40)$$

The cyclic shift property means that if $c(x)$ is a codeword, then the remainder of $xc(x)$ divided by $x^n + 1$ is also a codeword.

2.2 Generator Polynomial $g(x)$

Every cyclic code is uniquely defined by a **generator polynomial** $g(x)$ of degree $n - k$.

- $g(x)$ must be a factor of the polynomial $x^n + 1$ over $\text{GF}(2)$ (Galois Field of 2, modulo-2 arithmetic).
- A polynomial $c(x)$ of degree $\leq n - 1$ is a valid codeword if and only if it is a multiple of $g(x)$:

$$c(x) = m(x) \cdot g(x) \quad (41)$$

Where $m(x)$ is the message polynomial of degree $\leq k - 1$.

2.3 Systematic Encoding of Cyclic Codes

In systematic codes, the first k bits represent the message directly, and the remaining $n - k$ bits are parity check bits.

Systematic Cyclic Coding Steps

For message polynomial $m(x)$ and generator polynomial $g(x)$ of degree $n - k$:

Step 1: Multiply the message polynomial by x^{n-k} :

$$u(x) = x^{n-k}m(x)$$

Step 2: Divide $u(x)$ by $g(x)$ to obtain the remainder polynomial $r(x)$:

$$\frac{x^{n-k}m(x)}{g(x)} = q(x) + \frac{r(x)}{g(x)} \implies r(x) = \text{rem} \left[\frac{x^{n-k}m(x)}{g(x)} \right]$$

Step 3: Form the codeword polynomial $c(x)$ by adding the remainder $r(x)$ to $u(x)$ (addition is XOR in $\text{GF}(2)$):

$$c(x) = x^{n-k}m(x) + r(x) \quad (42)$$

Worked Example: (7, 4) Cyclic Systematic Code

Problem: For a (7, 4) cyclic code, the generator polynomial is $g(x) = x^3 + x + 1$. Encode the message vector $m = [1, 0, 1, 1]$ systematically.

Solution: Identify parameters: $n = 7$, $k = 4$, $n - k = 3$. Convert message vector to polynomial:

$$m(x) = 1 \cdot x^0 + 0 \cdot x^1 + 1 \cdot x^2 + 1 \cdot x^3 = 1 + x^2 + x^3$$

Multiply $m(x)$ by $x^{n-k} = x^3$:

$$x^3m(x) = x^3(1 + x^2 + x^3) = x^3 + x^5 + x^6$$

Divide $x^3m(x)$ by $g(x) = x^3 + x + 1$ using modulo-2 long division:

Divide: $x^6 + x^5 + x^3$ by $x^3 + x + 1$

$$x^6 + x^5 + x^3 = x^3(x^3 + x + 1) + x^5 + x^4 \quad (\text{since } x^6 + x^4 + x^3 \text{ subtracted leaves } x^5 + x^4)$$

Continuing long division gives:

$$\begin{array}{r} x^3 + x + 1 \overline{) x^6 + x^5 + 0x^4 + x^3} \\ \underline{x^6 + 0x^5 + x^4 + x^3} \quad (\text{multiply by } x^3) \\ x^5 + x^4 + 0x^3 \\ \underline{x^5 + 0x^4 + x^3 + x^2} \quad (\text{multiply by } x^2) \\ x^4 + x^3 + x^2 \\ \underline{x^4 + 0x^3 + x^2 + x} \quad (\text{multiply by } x) \\ x^3 + 0x^2 + x \\ \underline{x^3 + 0x^2 + x + 1} \quad (\text{multiply by } 1) \\ 1 \quad (\text{Remainder}) \end{array}$$

Thus, the remainder is $r(x) = 1$. Construct the codeword:

$$c(x) = x^3m(x) + r(x) = (x^6 + x^5 + x^3) + 1 = 1 + x^3 + x^5 + x^6$$

Convert polynomial to codeword vector:

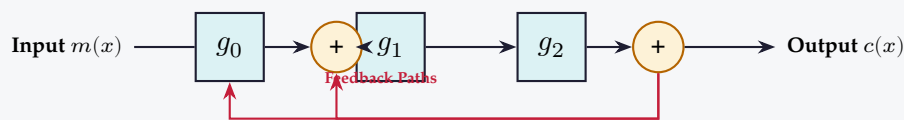
$$C = [1, 0, 0, 1, 0, 1, 1]$$

Where the first 3 bits $[1, 0, 0]$ represent the parity check bits, and the last 4 bits $[1, 0, 1, 1]$ represent the message.

2.4 Hardware Encoder using Feedback Shift Registers (FSR)

Cyclic codes are popular because their encoding and syndrome circuits can be implemented efficiently in hardware using shift registers and XOR gates (modulo-2 adders).

Feedback Shift Register Encoder for $g(x) = x^3 + x + 1$



3. Convolutional Codes

Linear block codes process data block-by-block. **Convolutional codes**, in contrast, process information continuously, using internal shift-register states to create memory dependencies.

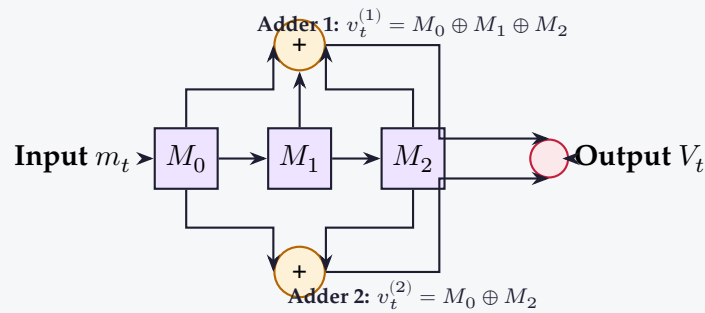
3.1 Basic Principles and Structure

A convolutional encoder is defined by three parameters (n, k, K) :

- n : Number of output bits produced in each shift cycle.
- k : Number of input bits processed in each shift cycle.
- K : Constraint length (number of shift register stages that affect the output).

The code rate is defined as $r = k/n$. The outputs are computed by modulo-2 addition of the current input and the values stored in the shift registers.

Convolutional Encoder Circuit (Rate 1/2, Constraint Length $K = 3$)

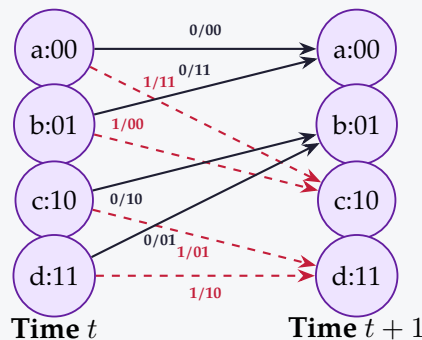


3.2 Representations of Convolutional Codes

Three graphical methods are used to describe a convolutional code:

1. **State Diagram:** Shows the states of the shift registers (excluding the current input) and the transitions between states caused by input '0' or '1'.
2. **Tree Diagram:** Shows all possible output paths as a branching tree structure, where the path branches upward for input '0' and downward for input '1'.
3. **Trellis Diagram:** A time-extended state transition diagram. It lists all possible states vertically at each time step, and shows transitions horizontally.

Trellis Diagram Transition Segment



3.3 Viterbi Decoding Algorithm

The Viterbi algorithm is the standard decoding method for convolutional codes. It finds the most likely sequence of transmitted symbols (minimum Hamming distance path) through the trellis.

- **Branch Metric:** The Hamming distance between the received bits and the expected output bits for a specific branch.
- **Path Metric:** The accumulated sum of branch metrics along a path.
- **Survivor Path:** At each node in the trellis, the incoming path with the lowest path metric is saved, and all other paths are discarded. This prevents exponential path growth.

4. Arithmetic Coding

Huffman coding assigns an integer number of bits to each symbol. If a symbol has a probability of 0.9, its ideal code length is $-\log_2 0.9 \approx 0.15$ bits, but Huffman must assign at least 1 bit, reducing efficiency. **Arithmetic coding** overcomes this constraint.

4.1 Core Concept

Arithmetic Coding: A non-block source coding algorithm that maps an entire sequence of symbols to a single fractional number in the interval $[0, 1)$. Symbols are represented by ranges proportional to their probabilities.

4.2 Encoding Procedure

Step 1: Initialize the interval boundaries: $\text{Low} = 0, \text{High} = 1$.

Step 2: For each symbol in the message:

$$\text{Range} = \text{High} - \text{Low} \quad (43)$$

$$\text{High} = \text{Low} + \text{Range} \times \text{Upper_Cum_Prob}(s_i) \quad (44)$$

$$\text{Low} = \text{Low} + \text{Range} \times \text{Lower_Cum_Prob}(s_i) \quad (45)$$

Step 3: Repeat for all symbols in the sequence.

Step 4: The final output is any floating-point number within the final $[\text{Low}, \text{High})$ interval.

Worked Example: Arithmetic Coding of "WET"

Problem: A source has three symbols $\{E, T, W\}$ with probabilities $\{0.5, 0.3, 0.2\}$.

- a. Calculate the cumulative probability ranges for the symbols.
- b. Encode the sequence "WET".

Solution:

(a) Probability Ranges:

Sort alphabet (say alphabetically $\{E, T, W\}$):

- Symbol E (prob = 0.5): Cumulative Interval $[0.0, 0.5)$
- Symbol T (prob = 0.3): Cumulative Interval $[0.5, 0.8)$
- Symbol W (prob = 0.2): Cumulative Interval $[0.8, 1.0)$

(b) Encode "WET":

- **Initial State:** $\text{Low} = 0, \text{High} = 1, \text{Range} = 1.0$.
- **First Symbol: 'W'** (Sub-interval $[0.8, 1.0)$):

$$\text{Low} = 0 + 1.0 \times 0.8 = 0.8$$

$$\text{High} = 0 + 1.0 \times 1.0 = 1.0$$

$$\text{Range} = 1.0 - 0.8 = 0.2$$

- **Second Symbol: 'E'** (Sub-interval $[0.0, 0.5)$ relative to current range):

$$\text{Low} = 0.8 + 0.2 \times 0.0 = 0.8$$

$$\text{High} = 0.8 + 0.2 \times 0.5 = 0.9$$

$$\text{Range} = 0.9 - 0.8 = 0.1$$

- **Third Symbol: 'T'** (Sub-interval $[0.5, 0.8)$ relative to current range):

$$\text{Low} = 0.8 + 0.1 \times 0.5 = 0.85$$

$$\text{High} = 0.8 + 0.1 \times 0.8 = 0.88$$

$$\text{Range} = 0.88 - 0.85 = 0.03$$

The final interval is $[0.85, 0.88)$. Any number in this range (e.g., **0.86**) represents the sequence "WET".

Quick Revision — Module III

Source & Channel Coding | PE-EC603D

Key Formulas at a Glance

Average Codeword Length	$L = \sum P_i l_i$ (bits/symbol)
Huffman Efficiency	$\eta = \frac{H(X)}{L} \times 100\%$
Cyclic Shift Shift	$C^{(1)} = (c_{n-1}, c_0, \dots, c_{n-2})$
Codeword Polynomial	$c(x) = m(x)g(x)$ (Non-systematic)
Systematic Codeword	$c(x) = x^{n-k}m(x) + r(x)$ where $r(x) = \text{rem} \left[\frac{x^{n-k}m(x)}{g(x)} \right]$
Syndrome Polynomial	$s(x) = \text{rem} \left[\frac{r(x)}{g(x)} \right]$ (If $s(x) = 0$, no error)
Convolutional Code Rate	$r = k/n$
Arithmetic Coding Range	Range = High – Low

One-Line Definitions

- **Huffman Coding:** A bottom-up optimal variable-length source coding technique based on sorting and merging probabilities.
- **Cyclic Code:** A linear block code where any cyclic shift of a codeword is also a valid codeword.
- **Generator Polynomial:** A unique polynomial $g(x)$ of degree $n - k$ that generates all codewords of a cyclic code.
- **Syndrome:** A polynomial remainder computed by the decoder to detect and locate errors in cyclic codes.
- **Constraint Length (K):** The number of shift register stages affecting output bits in a convolutional encoder.
- **Trellis Diagram:** A time-indexed state diagram representing paths and state transitions in convolutional codes.
- **Viterbi Decoder:** A maximum likelihood decoding algorithm using trellis paths to find the survivor path.
- **Arithmetic Coding:** A non-block coding method mapping a sequence of symbols to a single fractional number in $[0, 1)$.

Frequently Asked / Viva Questions

Q1. Why are Huffman codes uniquely decodable?

They possess the prefix-free property, meaning no codeword is a prefix of any other. This allows symbol-by-symbol decoding without ambiguity.

Q2. What is the difference between systematic and non-systematic codes?

In systematic codes, the message bits appear unchanged at the beginning (or end) of the codeword. In non-systematic codes, message and parity bits are mixed.

Q3. State the algebraic condition for a polynomial to be a generator polynomial of a cyclic code.

It must be of degree $n - k$ and must be a factor of $x^n + 1$ over Galois Field $GF(2)$.

Q4. How are parity bits calculated in systematic cyclic codes?

By dividing $x^{n-k}m(x)$ by $g(x)$ using modulo-2 division. The remainder of this division is the parity check polynomial $r(x)$.

Q5. What hardware components are required to build a cyclic code encoder?

Linear Feedback Shift Registers (LFSRs) consisting of D flip-flops, XOR gates (modulo-2 adders), and switches.

Q6. State the difference between block codes and convolutional codes.

Block codes process data in fixed, independent blocks. Convolutional codes process data continuously, utilizing internal register states to create memory dependencies.

Q7. What parameters define a convolutional code?

(n, k, K) , where n is output bits per cycle, k is input bits per cycle, and K is the constraint length.

Q8. Explain the role of the Viterbi algorithm in convolutional decoding.

It acts as a maximum-likelihood decoder, finding the path through the trellis that minimizes the Hamming distance (or Euclidean distance) to the received sequence.

Q9. What is a "survivor path" in Viterbi decoding?

At each trellis node, two paths enter. The path with the lower accumulated metric (errors) is kept as the survivor path, and the other is discarded.

Q10. Why does arithmetic coding outperform Huffman coding for highly skewed probability sources?

Huffman requires an integer number of bits per symbol, causing inefficiency for symbols with probabilities > 0.5 . Arithmetic coding represents sequences as a single fraction.

Q11. How does the receiver decode an arithmetic code?

The receiver uses the received fractional value to determine which symbol's sub-interval contains the value, outputs the symbol, updates the interval, and repeats.

Frequently Asked / Viva Questions (contd.)

Q12. What is the relationship between the degree of $g(x)$ and cyclic code parameters?

The degree of the generator polynomial $g(x)$ is always equal to $n - k$, which is the number of parity check bits in the (n, k) code.

Comparison of Coding Techniques

Parameter	Cyclic Codes	Convolutional Codes
Structure	Block-based (processes blocks of length k).	Continuous (processes streams of bits).
Memory	Memoryless between blocks.	Memory-based (states of shift registers).
Implementation	LFSR shift registers and XOR gates.	Shift registers and modulo-2 adders.
Decoding	Syndrome calculations.	Trellis-based Viterbi decoding.
Applications	Error detection (CRC in Ethernet, USB).	Deep space communications, cellular (LTE).